

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems

COURSE CODE: CSA0501

NAME: A. Suraj varma

REG.NO: 192525339

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems. (BL3, BL6)*

Activity Tool : Scenario-Based Task (High)

Assessment Tool Weightage: 35%

Dr

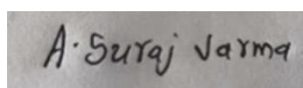
QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Given the scenario of a School Management System, identify relations and write SQL to list all students with their class teacher and grade.	BL3 – Apply	5
2	A hospital wants to track patient appointments. Design the relational schema and write SQL to find doctors with more than 10 appointments this month.	BL3 – Apply	5
3	In an e-commerce scenario, write SQL queries using sub-queries to find products that have never been ordered.	BL3 – Apply	5
4	For a university database, write relational algebra expressions to find students enrolled in both 'DBMS' and 'OS' courses.	BL3 – Apply	5
5	Given a payroll scenario, create a view showing employees whose salary is below department average and write a query to update their salary by 10%.	BL6 – Create	5
6	In a banking scenario, apply JOIN operations to retrieve customer account details, transaction history, and branch information.	BL3 – Apply	5

7	A retail store needs daily sales summary. Write SQL with GROUP BY and HAVING to report total sales per category exceeding Rs. 10,000.	BL3 – Apply	5
8	For a library system scenario, construct tuple relational calculus expressions to find members who borrowed books from all genres.	BL3 – Apply	5
9	Given an inventory management scenario, define CHECK and FOREIGN KEY constraints and demonstrate their enforcement through SQL.	BL3 – Apply	5
10	Design a relational schema for an HR system and create materialized views for monthly attendance and performance summaries.	BL6 – Create	5

Code of Conduct:

*I A. Suraj varma (192525339) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:



RUBRICS FOR CALCULATION:

Scenario based assessment					
Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues

Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Q1. Given the scenario of a School Management System, identify relations and write SQL to list all students with their class teacher and grade.

Relations

STUDENT

Column
StudentID (PK)
StudentName
ClassID (FK)
Grade

CLASS

Column
ClassID (PK)
ClassName
TeacherID (FK)

TEACHER

Column
TeacherID (PK)
TeacherName

SQL Query

```
SELECT s.StudentID,  
       s.StudentName,  
       t.TeacherName AS ClassTeacher,  
       s.Grade  
FROM Student s  
JOIN Class c  
ON s.ClassID = c.ClassID  
JOIN Teacher t  
ON c.TeacherID = t.TeacherID;
```

Output

StudentID	StudentName	ClassTeacher	Grade
S101	Ram	Mr. Kumar	A
S102	Nani	Ms. Priya	B
S103	Chaitan	Mr. Kumar	A+
S104	Rani	Ms. Priya	A

Explanation

The School Management System consists of three relations: **Student**, **Class**, and **Teacher**. The Student table is linked to the Class table using ClassID, and the Class table is linked to the Teacher table using TeacherID. The SQL query uses **INNER JOIN** to retrieve each student's details along with their class teacher and grade. This provides a complete list of students with their corresponding teacher and academic grade.

Q2. A hospital wants to track patient appointments. Design the relational schema and write SQL to find doctors with more than 10 appointments this month.

Relational Schema

DOCTOR

Attribute

DoctorID (PK)

DoctorName

Specialization

PATIENT

Attribute

PatientID (PK)

PatientName

Age

Gender

APPOINTMENT

Attribute

AppointmentID (PK)

PatientID (FK)

DoctorID (FK)

AppointmentDate

SQL Query

```
SELECT d.DoctorID,
       d.DoctorName,
       COUNT(a.AppointmentID) AS TotalAppointments
FROM Doctor d
JOIN Appointment a
ON d.DoctorID = a.DoctorID
WHERE MONTH(a.AppointmentDate) = MONTH(CURDATE())
AND YEAR(a.AppointmentDate) = YEAR(CURDATE())
GROUP BY d.DoctorID, d.DoctorName
HAVING COUNT(a.AppointmentID) > 10;
```

Sample Output

DoctorID	DoctorName	TotalAppointments
D101	Dr. Kumar	15
D102	Dr. Priya	12

Explanation

The hospital database contains three relations: **Doctor**, **Patient**, and **Appointment**. The **Appointment** table stores the appointment details of patients with doctors. The SQL query joins the **Doctor** and **Appointment** tables, counts the number of appointments for each doctor during the current month using COUNT(), groups the records using GROUP BY, and displays only those doctors who have **more than 10 appointments** using the HAVING clause.

Q3. In an e-commerce scenario, write SQL queries using sub-queries to find products that have never been ordered.

Relational Schema

PRODUCT

Attribute

ProductID (PK)

ProductName

Price

ORDERS

Attribute

OrderID (PK)

ProductID (FK)

OrderDate

Quantity

SQL Query (Using Sub-query)

```
SELECT ProductID, ProductName
FROM Product
WHERE ProductID NOT IN
(
    SELECT ProductID
    FROM Orders
);
```

Sample Output

ProductID	ProductName
P104	Smart Watch
P107	Bluetooth Speaker
P110	Laptop Bag

Explanation

The **Product** table stores the details of all products, and the **Orders** table stores the products that have been ordered by customers. The sub-query retrieves all **ProductID** values from the **Orders** table. The main query uses the **NOT IN** operator to display only those products whose **ProductID** is **not present** in the **Orders** table. Thus, the result shows the products that have **never been ordered**.

Q4. For a university database, write relational algebra expressions to find students enrolled in both 'DBMS' and 'OS' courses.

Relations

STUDENT

- StudentID
- StudentName

COURSE

- CourseID
- CourseName

ENROLLMENT

- StudentID
- CourseID

Relational Algebra Expression

DBMS_Students $\leftarrow \pi_{\text{StudentID}} (\sigma_{\text{CourseName} = \text{'DBMS'}} (\text{Student} \bowtie \text{Enrollment} \bowtie \text{Course}))$

OS_Students $\leftarrow \pi_{\text{StudentID}} (\sigma_{\text{CourseName} = \text{'OS'}} (\text{Student} \bowtie \text{Enrollment} \bowtie \text{Course}))$

Result $\leftarrow \text{DBMS_Students} \cap \text{OS_Students}$

Sample Output

StudentID
S101
S103
S105

Explanation

The relational algebra expression first retrieves the **StudentID** of students enrolled in the **DBMS** course and then retrieves the **StudentID** of students enrolled in the **OS** course. Finally, the **intersection** ($\cap$) operator is used to find the students who are enrolled in **both DBMS and OS** courses. This produces the list of students common to both course enrollments.

Q5. Given a payroll scenario, create a view showing employees whose salary is below department average and write a query to update their salary by 10%.

Relational Schema

EMPLOYEE

Attribute

EmployeeID (PK)

EmployeeName

DepartmentID (FK)

Salary

DEPARTMENT

Attribute

DepartmentID (PK)

DepartmentName

Step 1: Create View

```
CREATE VIEW LowSalaryEmployees AS
SELECT EmployeeID,
       EmployeeName,
       DepartmentID,
       Salary
FROM Employee e
WHERE Salary <
(
    SELECT AVG(Salary)
    FROM Employee
    WHERE DepartmentID = e.DepartmentID
);
```

Step 2: Update Salary by 10%

```
UPDATE Employee
SET Salary = Salary * 1.10
WHERE EmployeeID IN
(
```



```

SELECT EmployeeID
FROM LowSalaryEmployees
);

```

Sample Output

EmployeeID	EmployeeName	DepartmentID	Salary
E101	Ram	D01	33,000
E105	Priya	D02	29,700

Explanation

In the payroll system, the **Employee** table stores employee details and salaries, while the **Department** table stores department information. A **view** named **LowSalaryEmployees** is created to display employees whose salary is **below the average salary of their department**. The **UPDATE** query then increases the salary of these employees by **10%**. This approach helps organizations identify employees with lower salaries and update their pay fairly while using SQL views for efficient data management.

Q6. In a banking scenario, apply JOIN operations to retrieve customer account details, transaction history, and branch information.

Relational Schema

CUSTOMER

Attribute
CustomerID (PK)
CustomerName
AccountNo (FK)

ACCOUNT

Attribute
AccountNo (PK)
AccountType
Balance
BranchID (FK)

TRANSACTION

Attribute
TransactionID (PK)
AccountNo (FK)
TransactionDate
Amount

BRANCH

Attribute
BranchID (PK)
BranchName
Location

SQL Query (Using JOIN)

```
SELECT c.CustomerID,  
       c.CustomerName,  
       a.AccountNo,  
       a.AccountType,  
       t.TransactionDate,  
       t.Amount,  
       b.BranchName,  
       b.Location  
FROM Customer c  
JOIN Account a  
ON c.AccountNo = a.AccountNo  
JOIN Transaction t  
ON a.AccountNo = t.AccountNo  
JOIN Branch b  
ON a.BranchID = b.BranchID;
```

Sample Output

Customer ID	CustomerName	Account No	AccountType	Transaction Date	Amount	BranchName	Location
C101	Ram	A1001	Savings	05-08-2026	5000	Main Branch	Chennai
C102	Priya	A1002	Current	06-08-2026	12000	City Branch	Bangalore

Explanation

In the banking system, the **Customer**, **Account**, **Transaction**, and **Branch** tables are related through **AccountNo** and **BranchID**. The SQL query uses **INNER JOIN** to combine these tables and retrieve complete information, including customer details, account information, transaction history, and branch details. This helps the bank maintain a complete record of customer accounts and transactions while identifying the branch where each account is maintained.

Q7. A retail store needs daily sales summary. Write SQL with GROUP BY and HAVING to report total sales per category exceeding Rs. 10,000.

Relational Schema

CATEGORY

Attribute
CategoryID (PK)
CategoryName

PRODUCT

Attribute
ProductID (PK)
ProductName
CategoryID (FK)

SALES

Attribute
SaleID (PK)
ProductID (FK)
SaleDate
Quantity
TotalAmount

SQL Query

```
SELECT c.CategoryName,  
       SUM(s.TotalAmount) AS TotalSales  
FROM Category c  
JOIN Product p  
ON c.CategoryID = p.CategoryID  
JOIN Sales s  
ON p.ProductID = s.ProductID  
WHERE SaleDate = CURDATE()  
GROUP BY c.CategoryName  
HAVING SUM(s.TotalAmount) > 10000;
```

Sample Output

CategoryName	TotalSales
Electronics	25000
Groceries	14500
Clothing	12000

Explanation

In the retail store database, the **Category**, **Product**, and **Sales** tables are related through **CategoryID** and **ProductID**. The SQL query joins these tables, filters the sales for the **current day** using CURDATE(), groups the records by **CategoryName** using the GROUP BY clause, and calculates the total sales using SUM(TotalAmount). The HAVING clause displays only those categories whose total daily sales are **greater than Rs. 10,000**. This helps the retail store identify high-performing product categories for daily sales analysis.

Q8. For a library system scenario, construct Tuple Relational Calculus (TRC) expressions to find members who borrowed books from all genres.

Relational Schema

MEMBER

Attribute
MemberID (PK)
MemberName

BOOK

Attribute
BookID (PK)
BookTitle
Genre

BORROW

Attribute
BorrowID (PK)
MemberID (FK)
BookID (FK)
BorrowDate

Tuple Relational Calculus (TRC) Expression

$$\{ M \mid \text{MEMBER}(M) \wedge \\ \forall G (\exists B (\text{BOOK}(B) \wedge B.\text{Genre} = G.\text{Genre}) \\ \rightarrow \\ \exists R (\text{BORROW}(R) \wedge \\ R.\text{MemberID} = M.\text{MemberID} \wedge \\ R.\text{BookID} = B.\text{BookID})) \}$$

Sample Output

MemberID	MemberName
M101	Ram
M105	Priya

MemberID	MemberName
M108	Kiran

Explanation

In the library system, the **Member**, **Book**, and **Borrow** tables are related through **MemberID** and **BookID**. The Tuple Relational Calculus (TRC) expression identifies members who have borrowed books from **every available genre**. It uses the **universal quantifier (∀)** to ensure that for each genre, there exists a corresponding borrowed book by the member. This helps the library identify members who have read books across all genres, supporting comprehensive reading analysis.

Q9. Given an inventory management scenario, define CHECK and FOREIGN KEY constraints and demonstrate their enforcement through SQL.

Relational Schema

SUPPLIER

Attribute
SupplierID (PK)
SupplierName

PRODUCT

Attribute
ProductID (PK)
ProductName
Quantity
Price
SupplierID (FK)

SQL Code

Create Supplier Table

```
CREATE TABLE Supplier
(
    SupplierID INT PRIMARY KEY,
    SupplierName VARCHAR(50)
);
```

Create Product Table with CHECK and FOREIGN KEY Constraints

```
CREATE TABLE Product
(
    ProductID INT PRIMARY KEY,
    ProductName VARCHAR(50),
    Quantity INT CHECK (Quantity >= 0),
    Price DECIMAL(10,2) CHECK (Price > 0),
    SupplierID INT,
```

```
FOREIGN KEY (SupplierID)
REFERENCES Supplier(SupplierID)
);
```

Demonstration of Constraint Enforcement

Valid Data

```
INSERT INTO Supplier
VALUES (101, 'ABC Suppliers');
```

```
INSERT INTO Product
VALUES (1, 'Laptop', 20, 50000, 101);
```

Invalid Data (CHECK Constraint)

```
INSERT INTO Product
VALUES (2, 'Keyboard', -5, 1200, 101);
```

Result:

Error because **Quantity cannot be negative.**

Invalid Data (FOREIGN KEY Constraint)

```
INSERT INTO Product
VALUES (3, 'Mouse', 10, 800, 999);
```

Result: Error because **SupplierID 999 does not exist** in the **Supplier** table.

Explanation

In the inventory management system, the **Supplier** table stores supplier details, while the **Product** table stores product information. The **CHECK** constraint ensures that the **Quantity** is not negative and the **Price** is greater than zero. The **FOREIGN KEY** constraint maintains referential integrity by ensuring that every **SupplierID** in the **Product** table exists in the **Supplier** table. These constraints help maintain accurate and consistent inventory data.

Q10. Design a relational schema for an HR system and create materialized views for monthly attendance and performance summaries.

Relational Schema

EMPLOYEE

Attribute
EmployeeID (PK)
EmployeeName
Department
Designation

ATTENDANCE

Attribute
AttendanceID (PK)
EmployeeID (FK)
AttendanceDate
Status

PERFORMANCE

Attribute
PerformanceID (PK)
EmployeeID (FK)
PerformanceScore
ReviewMonth

Materialized View – Monthly Attendance Summary

```
CREATE MATERIALIZED VIEW Monthly_Attendance AS
SELECT EmployeeID,
       COUNT(*) AS TotalDaysPresent
FROM Attendance
WHERE Status = 'Present'
GROUP BY EmployeeID;
```

Materialized View – Monthly Performance Summary

```
CREATE MATERIALIZED VIEW Monthly_Performance AS
SELECT EmployeeID,
       AVG(PerformanceScore) AS AverageScore
FROM Performance
GROUP BY EmployeeID;
```

Sample Output

Monthly_Attendance

EmployeeID	TotalDaysPresent
E101	24
E102	26
E103	22

Monthly_Performance

EmployeeID	AverageScore
E101	88.5
E102	91.0
E103	84.7

Explanation

In the HR system, the **Employee**, **Attendance**, and **Performance** tables are related through **EmployeeID**. The **Monthly_Attendance** materialized view stores the total number of days each employee was present during the month. The **Monthly_Performance** materialized view stores the average performance score of each employee. Materialized views improve query performance by storing the computed results, making monthly attendance and performance reports faster and more efficient.